SET 1

PART A — Short questions and answers

Q1. Explain what an entity set is, using BOOK in a library database as an example.

Answer: An entity set is a group of similar real-world objects. BOOK is an entity set because it contains many books, each identified by Book_ID.

Q2. Compare a primary key and a foreign key using the BOOK_ISSUE and MEMBER tables.

Answer: A primary key uniquely identifies a row (for example, MEMBER.MemberID). A foreign key refers to a primary key in another table (for example, BOOK_ISSUE.MemberID).

Q3. Explain a functional dependency using Book_ID → Title.

Answer: Book_ID → Title means each Book_ID determines exactly one book title.

Q4. Compare INNER JOIN and LEFT OUTER JOIN with respect to unmatched rows.

Answer: INNER JOIN returns only matching rows. LEFT JOIN returns every row from the left table, with NULLs for unmatched right-side values.

Q5. Explain the purpose of the COUNT(*) function.

Answer: COUNT(*) counts all rows, including rows containing NULL values in some columns.

Q6. Explain the purpose of a stored procedure in PL/SQL.

Answer: A stored procedure is a named PL/SQL program saved in the database. It can accept parameters and perform reusable operations.

Q7. Compare a relational table with a MongoDB collection.

Answer: A relational table stores rows and columns with a fixed schema. A MongoDB collection stores flexible BSON documents.

Q8. Explain what a document is in MongoDB.

Answer: A MongoDB document is one record represented as field–value pairs, similar to a JSON object.

Q9. Interpret the MongoDB query: db.books.find({Category: "Computer Science"})

Answer: The query returns all documents that satisfy the specified filter condition.

Q10. Explain what a node and a relationship represent in a graph database.

Answer: A node represents an entity, and a relationship represents a connection between nodes.

PART B — System solution

Scenario: A city public library maintains a Library Management System to store member details, book details, and

1(a) ER design:

• **MEMBER**(MemberID, MemberName, MembershipType, Phone); primary key: **MemberID**.

• **BOOK**(BookID, Title, Author, Category, Price); primary key: **BookID**.

• **BOOK_ISSUE**(IssueID, MemberID, BookID, IssueDate, DueDate, ReturnDate, FineAmount); primary key: **IssueID**.

Connect the two main entity tables through the transaction/relationship table. The transaction table has its own primary key and foreign keys to both entity tables. Cardinality: each entity can participate in many transactions; each transaction belongs to one row from each entity table (1:M on both sides, resolving M:N). Include the transaction's date/status/amount attributes shown in the question.

1(b) SQL DDL/DML: Create each table with its listed columns, PRIMARY KEY on its ID, and FOREIGN KEYs in the transaction table. Add NOT NULL to required fields; use CHECK/UNIQUE/DEFAULT where the question asks. Insert five sample rows per table, making sure foreign-key values exist.

```
CREATE TABLE MEMBER (
  MemberID NUMBER PRIMARY KEY,
  MemberName VARCHAR2(100),
  MembershipType VARCHAR2(100),
  Phone VARCHAR2(100)
);
CREATE TABLE BOOK (
  BookID NUMBER PRIMARY KEY,
  Title VARCHAR2(100),
  Author VARCHAR2(100),
  Category VARCHAR2(100),
```

```
  Price NUMBER
);
CREATE TABLE BOOK_ISSUE (
  IssueID NUMBER PRIMARY KEY,
  MemberID NUMBER,
  BookID NUMBER,
  IssueDate DATE,
  DueDate DATE,
  ReturnDate DATE,
  FineAmount NUMBER
);
```

2(a) Join query: Select the two entity names/details from the transaction table by joining the transaction table to both entity tables using their IDs.

```
SELECT a.*, b.*
FROM BOOK_ISSUE x
JOIN MEMBER a ON x.MemberID = a.MemberID
JOIN BOOK b ON x.BookID = b.BookID;
```

2(b) Aggregate query: Group the transaction rows by the requested category/type and apply HAVING to the aggregate condition stated in the question.

```
SELECT grouping_column, COUNT(*) AS total
FROM transaction_table
GROUP BY grouping_column
HAVING COUNT(*) > threshold;
```

2(c) PL/SQL procedure: Create a procedure with an IN parameter for the requested ID. Join the transaction table to the related entity tables, filter by that parameter, and print selected values using DBMS_OUTPUT.PUT_LINE.

```
CREATE OR REPLACE PROCEDURE show_details(p_id IN NUMBER) AS
BEGIN
  FOR r IN (
    SELECT a.*, b.*, x.*
    FROM transaction_table x
    JOIN entity_table_1 a ON x.entity1_id = a.entity1_id
    JOIN entity_table_2 b ON x.entity2_id = b.entity2_id
    WHERE x.transaction_id = p_id
  ) LOOP
    DBMS_OUTPUT.PUT_LINE('Details retrieved');
  END LOOP;
END;
/
```

3. MongoDB: Create the collections and insert documents using the fields listed in the set. Query with find() and the requested filter.

```
db.collection.insertMany([
  { _id: 1, name: 'Sample A', category: 'Example', price: 500 },
  { _id: 2, name: 'Sample B', category: 'Example', price: 300 }
]);
db.collection.find({ author: 'Specified Author' });
db.collection.find({ price: { $gt: 400 } });
```